Dafny Power User:
Working with coinduction, extreme predicates, and ordinals

K. Rustan M. Leino and Jean-Baptiste Tristan

Manuscript KRML 285, 19 February 2023

Abstract. This note is a tutorial case study that formalizes and proves a simple result in abstract interpretation, with applications in compilation and program analysis. The formalization uses several advanced features in Dafny, including coinductive datatypes and least and greatest predicates (so-called *extreme* predicates) that we introduce along with some basic results in fixpoint theory. The presentation is aimed at users who have little or no prior experience with these features in Dafny, or who may be users of such features in other proof assistants.

0. Introduction

The purpose of this note is to show the use of two advanced features in Dafny: coinductive datatypes and extreme predicates. We do this by considering a concrete problem that is of importance in compiler construction and static analysis of programs and whose solution is an instance of abstract interpretation [1]. We choose this example because its underlying theory is the same as that of coinductive datatypes and extreme predicates: fixpoint theory. Note that the presentation that follows is informal, even though we use Dafny's syntax for clarity. The complete details will be provided in the note as needed.

Assume that we have some abstract type `A` equipped with some relation `le` and a constant `bot`

```
type A

predicate le(x: A, y: A)

const bot: A
```

We make the assumption that `bot` is smaller than any other element of `A`

```
lemma BotSmallest(x: A)
  ensures le(bot, x)
```

Let `F` be a monotonic function on `A`

```
function F(x: A): A

lemma FMonotonic(x: A, y: A)
  requires le(x, y)
  ensures le(F(x), F(y))
```

A fixpoint of `F` is a solution to the mathematical equation

```
  x = F(x)
```

We can define it as a Dafny predicate

```
predicate IsFixpoint(x: A) {
  x == F(x)
}
```

An important result in fixpoint theory, a variant of the famous Knaster-Tarski theorem [9], tells us that if `(A,le)` has no infinite ascending chains (which we will formalize later) and since `F` is monotonic, a fixpoint

exists and can be obtained iteratively from the value `bot`. Therefore, we would like to define the following function in Dafny:

```
function FindFixpoint(x: A): A {
  if IsFixpoint() then
    x
  else
    FindFixpoint(F(x))
}
```

where `FindFixpoint(bot)` computes the least fixpoint.

In practice, the type `A` is often defined as some abstract domain that captures some, but not all, the information about the actual values of a program. For example, one might consider an abstract domain where all we know about a variable is whether its content stays constant or not, whether it will eventually be used, or its possible range of values. In this context, `bot` correspond to the absence of information (before any collection of information has taken place) and `F` is a function that captures how the program transforms the abstract domain. For example, if the abstract domain is designed to track if a variable is constant and with what value, and assuming a statement of the form

```
y := x + 1;
```

if the abstract domain before the execution of this statement associates `x` with some constant `k`, then we can extend the abstract domain to include that `y` is also a constant, with value `k + 1`.

What makes this example particularly interesting to define in Dafny is that even though we know that the function terminates, we cannot define it as easily as we might like, precisely because we need to convince Dafny that it does terminate. Before we explain how to define such a function, it is useful to step back and ask: why do functions usually need to terminate in Dafny? The fundamental reason is that we want to ensure that we do not postulate the existence of functions that do not actually exist. For example, if we were to postulate the existence of a function `bad` from `nat` to `nat` that satisfies

```
bad(s)    ==    bad(s) + 1
```

then we can prove that `0 == 1`. The net result of this postulate is that Dafny's logic is now inconsistent.

One way to ensure that we do not postulate the existence of functions that do not actually exist is to ensure that they terminate on all of their inputs. In most cases, this can be done by hinting to Dafny (using a `decreases` clause) that there is a well-founded order and that recursive calls yield progressively smaller values in that order. But in the case of function `FindFixpoint`, the problem is more abstract and we know that it terminates because of some abstract mathematical theorem. Even if we were to prove this theorem in Dafny, then we might be able to show that the function `FindFixpoint` exists, but that is not good enough: we want to be able to define this function in such a way that it can be compiled to actual code and executed.

Note that termination is just a means to an end: we know that functions that terminate will not make the logic inconsistent. There are other ways to define functions and guarantee that they are consistent, for example by defining them as… fixpoints! Thankfully, Dafny has everything we need to do this for predicates. To define our function, we will use two key ideas.

First, function `FindFixpoint` will not return the fixpoint. Instead, we will use a coinductive datatype to define a potentially infinite sequence of values, also known as a stream, and define `FindFixpoint` as producing the sequence of its iterates

```
x, F(x), F(F(x)), F^3(x), F^4(x), ...
```

Second, we will show how we can work and reason over such potentially infinite streams using extreme predicates. As mentioned previously, the theory underlying coinductive datatypes and extreme predicates is the same as the theory behind the termination of function `FindFixpoint` and we will now introduce just enough to provide a solid understanding of these features and the solution to our problem.

0.0. Prerequisites

In this note, we assume that you have some familiarity with Dafny. In particular, you should have some experience defining and using predicates, functions, lemmas, datatypes, type parameters, type parameter characteristics, the ghost modifier, and Skolemization. This information can be found in the reference manual and previous Dafny Power User notes.

1. Background

1.0. The Knaster-Tarski Theorem

To understand coinductive datatypes and extreme predicates, it is useful to understand a special case of the Knaster-Tarski theorem [9]. Recall that the powerset of a set U, written 2^U, is the set of all subsets of U. It contains at least the empty set $\emptyset$ and the set U itself and can be equipped with all the usual set operations: union ($\cup$), intersection ($\cap$), and set inclusion $\subseteq$. Moreover, it has the following properties:

- Set inclusion $\subseteq$ makes 2^U a partially ordered set
- $\emptyset$ is smaller than any other set in 2^U
- U is greater than any other set in 2^U
- $A \cup B$ is the smallest set that contains the elements of A and the elements of B
- $A \cap B$ is the largest set that contains the elements that belong to both A and B

The Knaster-Tarski theorem implies that for any set U and monotonic function $F : 2^U \rightarrow 2^U$, F has a unique least fixpoint (that is, smaller than any other fixpoint) defined as

$$\cap \{ x \in 2^U \mid F(x) \subseteq x \}$$

and a unique greatest fixpoint (that is, bigger than any other fixpoint) defined as

$$\cup \{ x \in 2^U \mid x \subseteq F(x) \}$$

1.1. Coinductive Datatypes

Before we explain coinductive datatypes, it is useful to review the classic inductive datatypes that you should be familiar with. When we define a datatype such as

```
datatype List<X> = Nil | Cons(head: X, tail: List<X>)
```

where `X` is a type parameter, we usually intuitively interpret the type `List<X>` as the set of all finite lists that carry values of type `X`, it is useful to take a more formal point of view. The constructors `Nil` and `Cons` introduce an alphabet, and the typing rules a grammar. These induce a set of possible strings. Let U be the set of all such valid strings. By defining `List<X>` as a datatype, we define a monotonic function `F` from 2^U to 2^U

```
F(S)  =  {Nil} + {Cons(v,l) | v in X and l in S}
```

and also specify that out of all the possible solutions of this equation, if any, we are interested in the smallest one. By the Knaster-Tarski theorem, we indeed know that the least fixpoint of `F` exists and is unique, and we define type `List<X>` as

$$\cap \{ x \in 2^U \mid F(x) \subseteq x \}$$

and it is possible to prove that this corresponds to the natural interpretation of `List<X>` as the set of all finite lists where we know that `Nil` belongs to that set, and that if some value `l` belongs to that set, so does `Cons(v,l)` for any value `v` of type `X`.

A coinductive datatype is defined essentially like an inductive one, with the keyword `codatatype` instead of `datatype`.

```
codatatype Stream<X> = Nil | Cons(head: X, tail: Stream<X>)
```

Coinductive datatypes also define a monotonic function `F` from 2^U to 2^U, in this case the same one as `List<X>`

```
F(S)  =  {Nil} + {Cons(v,l) | v in X and l in S}
```

However, in this case, we are interested in the largest solution to this equation. Again, by the Knaster-Tarski theorem, we know that the greatest fixpoint of `F` exists and is unique, and we define type `Stream<X>` as

$$\cup \{ x \in 2^U \mid x \subseteq F(x) \}$$

and it is possible to prove that this set contains not only the finite values, but also all the infinite values.

In conclusion, even though the definitions of `List` and `Stream` use the same constructors and define the same monotonic function `F`, we interpret them differently as either the least fixpoint of `F` (datatypes) or the greatest fixpoint of `F` (codatatypes). In general, the difference between these two interpretations is that the datatype will only contain finite values, whereas the codatatype will also contain infinite values.

Finally, an important point of terminology. The codatatype `Stream` that we use in this note should not be considered as the only possible definition of the concept of a stream. The concept of a stream takes on slightly different interpretation in object-oriented programming, reactive programming, functional programming, or CS theory.

1.2. Extreme Predicates

Recall that when we define a recursive predicate, we are implicitly postulating its existence. Obviously, to prevent logical inconsistencies, we want to make sure that such predicates actually do exist. One way to do this is to ensure that the predicate always terminates. In some cases, this restriction is too strong. For example, we would like to be able to define the following predicate, which even though it does not terminate, postulates the existence of a function that does exist:

```
predicate IsFinite(s: Stream) {
  s == Nil || IsFinite(s.tail)  // error: recursive call may not terminate
}
```

The Knaster-Tarski theorem provides us with an alternative way to define recursive predicates while ensuring consistency of the logic. To understand why, recall that a predicate $P(x)$ on some domain X can be understood as the set of values for which it is true: $\{ x \in X \mid P(x) \}$. The definition of a recursive predicate can be understood not as a predicate directly, but first as a function from 2^X to 2^X, and it is monotonic.

```
IsFinite(S) == {Nil} + {Cons(v,l) | v in X and l in IsFinite(S)}
```

By the Knaster-Tarski theorem, we know that this equation has both a unique least fixpoint and a unique greatest fixpoint, and these sets correspond to predicates that exist and can be used without creating inconsistencies. In the same way that we use two keywords, `datatype` and `codatatype`, to indicate whether we are interested in the least predicate or greatest predicate fixpoint solution, we use two keyword phrases, `least predicate` and `greatest predicate` to denote which solution we are interested in. In conclusion, we can define predicate `IsFinite` as

```
least predicate IsFinite(s: Stream) { s == Nil || IsFinite(s.tail) }
```

1.3. Ordinal Numbers and Transfinite Recursion/Induction

Now that we have seen how to define coinductive datatypes and extreme predicates, we need to explain how to effectively work with them, in particular to define executable code. To do this, we will introduce a constructive variant of the Knaster-Tarski theorem. This theorem makes use of the concept of ordinal numbers and we need to introduce them. Ordinals are a somewhat abstract topic and it would not be appropriate to provide a comprehensive and rigorous introduction in these notes. Instead, we will try to provide an intuitive understanding of what ordinals are, and concretely explain how to use them.

Recall that natural numbers have two fundamental usages: they can be used to count (cardinal numbers), but also to order (ordinal numbers). This is reflected in English with cardinals being named zero, one, two, etc… while ordinals are named first, second, third, etc… As long as we are counting or ordering a finite number of objects, there is no difference between cardinals and ordinals, and we think mostly in terms of the more abstract concept of natural number. However, once you start counting or ordering an infinite number of objects, cardinals and ordinals start to have a very different behavior.

In Dafny, you can effectively use ordinals with only a very limited amount of knowledge about them. In fact, Dafny itself has a very limited amount of knowledge about ordinals. Nevertheless, it can be difficult (and unpleasant) to work with a concept for which we have no intuition whatsover, so we will attempt to provide some explanation based on the work of Per Martin-L"{o}f. [8]

All natural numbers are ordinal numbers, including 0. To use analogy, you can think of these ordinals as waiting lines to some goal. 0 is a waiting line with noone in it. 1 is a waiting line with one person in it, etc… Now, imagine that to get to your goal, you have to first go through the waiting line with noone in it (0), then through the waiting line with one person in it (1), then through the waiting line with two persons in it (2), and so on, for every possible line that corresponds to a natural number. This new “meta” line is made of an infinite number of lines, and this is the ordinal number ω. You could also consider a waiting line where you start with the ω waiting line, followed by a waiting line with one person, and so on… When we take existing waiting lines to create a “meta” line like we did for ω, we are creating what is called a *limit* ordinal, written in general using the letter λ. Note that we have three different kinds of waiting lines corresponding to three important categories of ordinals: 0, limit ordinals, and all the other ordinals such as 3, $\omega + 3$, $2\ \omega + 4$, etc… Actually, there are only two cases, because 0 can be viewed as a limit ordinal (namely, the limit of the empty set of things that precede it). An ordinal that is not a limit ordinal is called a *successor* ordinal.

Again, you don't need to know much more about ordinals to use them in Dafny. To summarize, every ordinal has the form `λ + n`, where `λ` is a *limit ordinal* and `n` is a natural number. The smallest limit ordinal is `ω`, so the ordinals include all the natural numbers. Indeed, the natural numbers are the smallest ordinals, and the smallest ordinal that is not a natural number is in mathematics called `ω`. In layman's terms, `ω` is known as “infinity”, but the there are far larger ordinals than this “infinity”. Here's a small visualization of the “ordinal number line” that may be helpful in understanding ordinals:

```
0, 1, 2, ..., ω, ω+1, ω+2, ..., ω*2, ω*2+1, ω*2+2, ..., λ, λ+1, λ+2, ...
```

Remark 0.
Dafny doesn't actually know many properties of the built-in type `ORDINAL`. It only knows enough to enable working with extreme predicates and their prefix predicates. In fact, there's not even any syntax for writing down ordinals other than the ordinals that are natural numbers. The notation `ω` and `λ` we used above is not Dafny syntax.

In the same way that it is possible to define a function by recursion over the naturals or prove a property by induction over the naturals, it is possible to define a function by recursion over the ordinals and prove properties by induction over the ordinals. These recursion/induction principles generalize that of the natural and you need to consider three cases: 0, limit ordinals, and all the other ordinals that have predecessors. Unlike with naturals where every decreasing sequence ends at 0 in the worst-case, ordinals can decrease to any limit ordinal, and this is why we need to define a function or prove a property by considering not just 0 as

a base case, but every possible limit ordinal. That is, the ordinals are *well-founded*, and really this is all we need to know about them in principle. In practice, you need to know that there are 3 instead of two cases to consider, and we will show a detailed example later in that note. Finally, a point of terminology: recursion and induction on ordinals are called transfinite recursion and transfinite induction.

1.4. The Constructive Knaster-Tarski Theorem

The Knaster-Tarski theorem tells us under what conditions a fixpoint exists, and even characterizes the least and greatest fixpoints. This is key to the introduction of codatatypes and extreme predicates in the Dafny language, but of no help to a Dafny programmer who wants to effectively compute a fixpoint. Fortunately, there is a constructive version of the Knaster-Tarski theorem [0]. Stating it formally would require too much formalism, but in essence, it states that if `F` is a monotonic function, then the least fixpoint of `F` can be obtained by iteration of `F` over the ordinals.

Our explanations will once again use the definition of `IsFinite`

```
least predicate IsFinite(s: Stream) {
  s == Nil || IsFinite(s.tail)
}
```

which can be understood as a monotonic recursive function from 2^X to 2^X

```
  IsFinite(S) == {Nil} + {Cons(v,l) | v in X and l in IsFinite(S)}
```

which itself corresponds to the proposition

```
IsFinite(s)                <==>    s == Nil || IsFinite(s.tail)
```

In order to express transfinite iterations (or inductions for proofs), Dafny defines a *prefix predicate* `IsFinite#`. The prefix predicate takes one more parameter than the least predicate does, and this parameter can be understood as an upper bound on the number of times the left-hand side of the recurrence equation can be rewritten into the right-hand side. This is easier to understand in symbols. For any `n`, we have

```
IsFinite#^0(s)             <==>    false
IsFinite#^(n+1)(s)         <==>    s == Nil || IsFinite#^n(s.tail)
```

Here, we've written the extra parameter as a superscript, to suggest applying the function some number of times. For a limit ordinal `λ`, we define `IsFinite#`$^\lambda$ as the disjunction over all smaller ordinals. Here's our new definition, where `o` is any ordinal and `λ` is any limit ordinal:

```
IsFinite#^0(s)             <==>    false
IsFinite#^(o+1)(s)         <==>    s == Nil || IsFinite#^o(s.tail)
IsFinite#^λ(s)             <==>    ∨_{o < λ} (s == Nil || IsFinite#^o(s.tail))
```

Since `0` is a limit ordinal and there is no ordinal that's smaller, the first line is a special case of the third line. Also, it is useful to realize that another way to write a large disjunction is to write `∃` instead of `∨`. Thus, we can use a more Dafny-like notation to write the definition of `IsFinite#`:

```
IsFinite#^(o+1)(s)         <==>    s == Nil || IsFinite#^o(s.tail)
IsFinite#^λ(s)             <==>    exists o :: o < λ && (s == Nil || IsFinite#^o(s.tail))
```

Using the same notation to write `IsFinite` in terms of `IsFinite#`, we have

```
IsFinite(s)    <==>    exists o :: IsFinite#^o(s)
```

2. Streams

The type `Stream`, parameterized by a type `X` that stands for the payload of the stream, has a familiar shape:

```
codatatype Stream<X> = Nil | Cons(head: X, tail: Stream<X>)
```

In order to solve our simple abstract interpretation problem, we will need but the simplest stream operations:

- A predicate to distinguish finite from infinite streams
- A function that computes the Length of a finite stream
- A function that returns the last value of a nonempty, finite stream

2.0. Distinguishing Finite and Infinite Streams

A stream is finite if it eventually contains `Nil`; otherwise, it is infinite. As explained previously, if we were to attempt to define this predicate as a standard predicate

```
predicate IsFinite(s: Stream) {
  s == Nil || IsFinite(s.tail)  // error: recursive call may not terminate
}
```

Dafny would report an error, because it's unable to prove termination of the recursive call. Indeed, if `s` is an infinite stream, the recursive call would fail to terminate. Therefore, predicate `IsFinite` must be defined as an extreme predicate:

```
least predicate IsFinite(s: Stream) {
  s == Nil || IsFinite(s.tail)
}
```

Exercise 0.
Definite `IsFinite'` like `IsFinite` above, but using `greatest predicate`. Then, prove that `IsFinite'(s)` holds for any stream `s` (which strongly suggests that `IsFinite'` does not match our understanding of what being finite means). You may need to read more of this note before you know how to prove this property, but we give the answer in Appendix A.

2.1. Computing the Length of a Finite Stream

If a stream is finite, we would like to be able to compute its length. This is not as simple as it might seem at first glance, and provides an interesting use case to better understand the use of ordinals. The specification of the `Length` function is given by

```
function Length(s: Stream): nat
  requires IsFinite(s)
```

It uses the predicate `IsFinite` as part of its *precondition*, as indicated by the `requires` keyword. Since Dafny enforces preconditions at call sites, there is no way that `Length` could ever be invoked on an infinite stream.

We're not going to use the body of `Length` directly. Instead, we will use the properties of `Length` that are stated by the following lemma:

```
lemma AboutLength(s: Stream)
  requires s != Nil && IsFinite(s)
  ensures Length(s) == 1 + Length(s.tail)
```

For a nonempty, finite stream `s`, this lemma says that `s.tail` is also finite and that `Length(s)` is `1` more than `Length(s.tail)`. To obtain these properties, simply call the lemma. In fact, in Dafny, you *have* to call the lemma explicitly to make use of what it states—there is no “automatically applied” lemma, as there is in some other proof assistants (see Dafny Power User note [6] for more information).

A first attempt at defining `Length` might be

```
function Length(s: Stream): nat
  requires IsFinite(s)
{
  if s == Nil then 0 else 1 + Length(s.tail) // error: cannot prove termination
}
```

Alas, despite the precondition `IsFinite(s)`, the termination proof for the recursive call does not go through. To sort this out will take some effort.

2.1.0. Defining `Length'` from a prefix predicate

The problem with writing a straightforward definition of `Length` is that we need to prove termination. Since for the precondition `IsFinite(s)` there is an ordinal `k` such that `IsFinite#`^{k}, we can hope to use that `k` to prove termination.

Our next attempt is therefore to define a function `Length'` as follows:

```
function Length'(ghost k: ORDINAL, s: Stream): nat
  requires IsFinite#[k](s)
{
  if s == Nil then
    0
  else if k.Offset != 0 then
    1 + Length'(k - 1, s.tail)
  else
    var k' :| k' < k && IsFinite#[k'](s);
    Length'(k', s)
}
```

There are several things to explain.

In our explanations in the background section, we notated the extra parameter of the prefix predicate as a superscript. In Dafny syntax, the extra parameter also has a special notation, but rather than putting it in a superscript, it is given in square brackets. You see this in the precondition of `Length'`.

We intend the function to be one that can be compiled and run, but the parameter `k` is used only in the proof of termination. Therefore, we declare `k` to be a ghost parameter. All specifications in Dafny are ghost, so they don't cost anything at run time. Moreover, lemmas, extreme predicates, and prefix predicates are always ghost.

After the `s == Nil` case, we proceed according to the two cases of the definition of the prefix predicate `IsFinite#`. That is, we take one action if `k` is not a limit ordinal and another action if `k` is a limit ordinal. Since every ordinal has the form `λ + n` for some limit ordinal `λ` and natural number `n`, we can tell if an ordinal is a limit ordinal or not by inspecting the `n` part. In Dafny, you obtain this `n` part using the member `Offset` of an ordinal. Thus, the test `k.Offset != 0` says that `k` is not a limit ordinal.

In the final case (where `k` is limit ordinal), the definition of `IsFinite#`^{k} tells us that there is a smaller ordinal `k'` for which `IsFinite#`$^{k'}$ holds. To obtain the smaller ordinal, we introduce a local variable `k'` and (using the assign-such-that operator `:|`) assign to it a value such that the condition

```
k' < k && IsFinite#[k'](s)
```

holds. There is a proof obligation that a value for such a `k'` exists, but that proof obligation follows directly from the definition of `IsFinite#` and the fact that `k` is a limit ordinal whenever the last `else` branch is taken.

Our definition of `Length'` is straightforward but problematic. The function's body uses `k`, which is ghost. Moreover, even if we changed `k` to make it be a non-ghost (that is, compiled) parameter, then the use of `IsFinite#` in the assignment to `k'` makes `k'` ghost, so we can't use it in a compiled function body. If we change `Length'` to be a ghost function, like:

```
ghost function Length'(k: ORDINAL, s: Stream): nat
```

then the definition we gave is fine. For the purposes of the program-analysis theorem we will prove in this note, we don't need the `Length` function to be compiled, so this would be alright. However, for a general library for streams, we really ought to provide a compiled `Length` function.

So, let's rewrite the body of `Length'`.

2.1.1. Reducing `k` to a sufficiently small value

We can write a compiling body for `Length'` from the realization that any recursion in our first attempt only reduces the value of `k`. Let's do that in a separate function, `ReduceK`. The idea is that `ReduceK(k, s)` first follows smaller limit ordinals that don't affect the value of `IsFinite#[k]`, and then returns `k - 1`.

```
ghost function ReduceK(k: ORDINAL, s: Stream): (k': ORDINAL)
  requires s != Nil && IsFinite#[k](s)
  ensures k' < k && IsFinite#[k'](s.tail)
{
  if k.Offset != 0 then
    k - 1
  else
    var k' :| k' < k && IsFinite#[k'](s);
    ReduceK(k', s)
}
```

There are several things to observe about this function. First, since we don't intend for the function to present at run time, we declare it as ghost.

Since functions in Dafny are transparent—meaning, their bodies are available to callers—most functions do not have a declared postcondition. For a recursive function, like `ReduceK`, a property about the function's result may not be immediately obvious from the function body. In those cases, a postcondition can be a good idea, because the proof of the postcondition can then make use of the postcondition of any recursive call.

One way for `ReduceK` to mention its result in the postcondition is to write `ReduceK(k, s)`, that is, the function itself with the given arguments. However, since our postcondition above needs to mention the result twice, it is more convenient to give the result value a name. That's why we introduced the name `k'` in the function signature. (The identifier `k'` can only be used in the postcondition of the function.)

Using `ReduceK`, we can rewrite `Length'` as a compiled function that uses `k` to prove termination but does not need `k` at run time:

```
function Length'(ghost k: ORDINAL, s: Stream): nat
  requires IsFinite#[k](s)
{
  if s == Nil then
    0
  else
    1 + Length'(ReduceK(k, s), s.tail)
}
```

And with `Length'` in place, we define `Length`:

```
function Length(s: Stream): nat
  requires IsFinite(s)
{
  var k :| IsFinite#[k](s);
  Length'(k, s)
}
```

Since the prefix predicate `IsFinite#` is ghost, the variable `k` is automatically made a ghost. If we wanted to make this more explicit in the program text, we can write

```
  ghost var k :| IsFinite#[k](s);
```

However, there is an issue with assign-such-that construct in the function. The issue won't be apparent until we start proving properties about the function, but since we know it's coming, we'll address it right away. As used here, the `:|` operator has many possible values it can pick for `k`. Which one you get it underspecified, but Dafny promises to be consistent about which value it picks. This is necessary in order for expressions to be deterministic (see [5] for more information). More precisely, Dafny promises to be consistent for each textual occurrence of `:|`. This means that if you write this very same `:|` construct somewhere else, like in a proof

about `Length`, then the other `:|` operator may pick a different `k`. To prevent this, we need to make sure that these use the same textual occurrence of `:|`, so we put it in a function:

```
function Length(s: Stream): nat
  requires IsFinite(s)
{
  ghost var k := PickK(s);
  Length'(k, s)
}

ghost function PickK(s: Stream): ORDINAL
  requires IsFinite(s)
{
  var k :| IsFinite#[k](s); k
}
```

2.1.2. Proving the lemma about `Length`

Finally, we can prove the lemma about `Length`:

```
lemma AboutLength(s: Stream)
  requires s != Nil && IsFinite(s)
  ensures Length(s) == 1 + Length(s.tail)
```

In the previous section, we addressed the termination of `Length` by explicitly keeping track of the prefix-predicate index `k`. We added an auxiliary function, `Length'`, and let that function take `k` as a parameter. Let's consider what happens when `Length` is called for a non-`Nil` stream. The call `Length(s)` will call `Length'(PickK(s), s)`, which in turn will call

```
Length'(ReduceK(PickK(s), s), s.tail)
```

In the postcondition of the lemma, we also have to reason about `Length(s.tail)`. When it gets called, it will end up calling

```
Length'(PickK(s.tail), s.tail)
```

If we're going to show a connection between `Length(s)` and `Length(s.tail)`, we would wish that the first parameter to the two `Length'(..., s.tail)` calls above are the same. Unfortunately, we have no such guarantee. Fortunately, we can prove a lemma that says the value of the first parameter does not matter—provided the value is one that is allowed by the precondition:

```
lemma Length'AnyK(k0: ORDINAL, k1: ORDINAL, s: Stream)
  requires IsFinite#[k0](s) && IsFinite#[k1](s)
  ensures Length'(k0, s) == Length'(k1, s)
{
}
```

This lemma is proved automatically (using Dafny's automatic induction), so there is nothing we need to write between the curly braces of the body of this lemma.

Using the `Length'AnyK` lemma, the proof of `AboutLength` is straightforward:

```
lemma AboutLength(s: Stream)
  requires s != Nil && IsFinite(s)
  ensures Length(s) == 1 + Length(s.tail)
{
  calc {
    Length(s);
  ==  // def. Length
    Length'(PickK(s), s);
  ==  // def. Length'
    1 + Length'(ReduceK(PickK(s), s), s.tail);
  ==  { Length'AnyK(ReduceK(PickK(s), s), PickK(s.tail), s.tail); }
    1 + Length'(PickK(s.tail), s.tail);
  ==  // def. Length
    1 + Length(s.tail);
  }
}
```

This proof uses a proof calculation ("the `calc` statement" [7]). The steps of the proof calculation are proved automatically, except the step where we need to call the `Length'AnyK` lemma.

A shorter way to write the proof of `AboutLength` is to include just the call to the salient lemma:

```
lemma AboutLength(s: Stream)
  requires s != Nil && IsFinite(s)
  ensures Length(s) == 1 + Length(s.tail)
{
  Length'AnyK(ReduceK(PickK(s), s), PickK(s.tail), s.tail);
}
```

This proof is shorter, but perhaps less illuminating to a human reader. Dafny is happy with either, so take your pick.

2.2. Returning the Last Value of a Finite Stream

Lastly, we define a function that returns the last element of a nonempty, finite stream:

```
function Last<X>(s: Stream<X>): X
  requires s != Nil && IsFinite(s)
  decreases Length(s)
{
  match s
  case Cons(x, Nil) =>
    x
  case _ =>
    AboutLength(s);
    Last(s.tail)
}
```

3. Application to Simple Abstract Interpretation

3.0. Problem Statement

We assume an opaque type `A` to stand for the elements of our abstract domain.

```
type A(==,0,!new)
```

Recall that a type declaration can be equipped with *type characteristics*, and in this case, we require type `A` to support equality comparison `(==)`, to be nonempty `(0)`, and to be in the purely functional fragment of Dafny, without references `(!new)`.

Type `A` is equipped with a predicate `le`. For now, we do not make any assumptions about this predicate, but intuitively, it will induce some ordering on `A` and can be read as *less or equal*

```
predicate le(x: A, y: A)
```

We can define its counterpart, *less than*

```
predicate lt(x: A, y: A) {
  le(x, y) && x != y
}
```

We need `A` to have a bottom element that is smaller than any other element. To that avail, we declare a constant `bot` and state the property of `bot` as a lemma:

```
const bot: A

lemma BotSmallest(x: A)
  ensures le(bot, x)
```

We omit the body of this lemma, since we don't intend to prove it. In effect, this makes the lemma an axiom.

Remark 1.
To use this abstract library and write code that compiles and runs, you will have to refine `A` into a concrete type, define `bot`, and prove `BotSmallest`.

Finally, we assume the existence of a monotonic function on `A`. Let's call it `F` and declare it here:

```
function F(x: A): A

lemma FMonotonic(x: A, y: A)
 requires le(x, y)
    ensures le(F(x), F(y))
```

Our goal is to define a function `iterates` that will effectively compute the least fixpoint of `F`, starting from `bot`, where the fixpoint is formally defined as

```
predicate IsFixpoint(x: A) {
  x == F(x)
}
```

3.1. Ascending Chain Condition

We do need to make some assumptions about `le` to guarantee the existence of a fixpoint. In general, `le` is expected to be a partial order, but in this simple example, all we need about `(A, le)` is that it has no infinite ascending sequences. We define what it means for all ascending sequences starting at an element `x` to be finite. This is elegantly stated using a least predicate:

```
least predicate Acc(x: A) {
  forall y :: lt(x, y) ==> Acc(y)
}
```

Here's a way to think about this predicate. For any maximal element `x`, the quantifier is trivially `true`. Thus, `Acc(x)` holds for any such `x`. Also, for any element `x` whose only larger elements are maximal elements, `Acc(x)` holds. More generally, the definition says that `Acc(x)` holds for any element `x` whose only larger elements are ones that we have already determined to satisfy `Acc`. But since we declared `Acc` to be a *least* predicate, the “already determined” means “determined in a finite number of steps”. For all other elements `x`, `Acc(x)` is `false`.

We state as an axiom that all ascending sequences from `bot` are finite:

```
lemma BotAcc()
  ensures Acc(bot)
```

Remark 2.
From `Acc(bot)`, it follows that every ascending sequence in `A` is finite. Here's a proof thereof:

```
lemma WellFounded(x: A)
  ensures Acc(x)
{
  calc {
    true;
  ==> { BotSmallest(x); }
    le(bot, x);
  ==> // def. lt
    bot == x || lt(bot, x);
  ==> { BotAcc(); }
    Acc(x);
  }
}
```

The last step of this proof calculation also uses the definition of `Acc`. Since Dafny automatically unfolds functions and predicates once, we don't need to mention that fact.

The proof calculation is quite readable, but with less pomp and circumstance, it also possible to prove the lemma by just calling other other two lemmas:

```
  BotSmallest(x);
  BotAcc();
```

3.2. Ascending Streams

With the ordering on `A`, we can define what it means for a stream to be ascending. We'll use a greatest predicate for that:

```
greatest predicate Ascending(s: Stream<A>) {
  match s
  case Nil => true
  case Cons(x, Nil) => true
  case Cons(x, Cons(y, _)) => lt(x, y) && Ascending(s.tail)
}
```

Since we chose a greatest predicate, this definition considers both finite and infinite streams to be ascending, provided any adjacent elements `x` and `y` satisfy `lt(x, y)`.

Exercise 1.
Declare a predicate `FiniteAndAscending(s: Stream<A>)`

In a lattice that satisfies the ascending chain condition, there are no infinite ascending streams. If we want to prove a lemma whose antecedent contains a least predicate, then we can use a `least lemma`. That provides some scaffolding that lets us focus on the main ingredients of the proof. Dually, if we want to prove a lemma whose conclusion contains a greatest predicate, then we can use a `greatest lemma` (that's a hint for doing Exercise 0).

Here we go:

```
least lemma AscendingIsFinite(s: Stream<A>)
  requires (s == Nil || Acc(s.head)) && Ascending(s)
  ensures IsFinite(s)
{
}
```

Dafny completes this proof automatically!

That was too easy. Let's manually introduce more and more detail into this proof to see what Dafny does under the hood. To start with, let's turn off Dafny's automatic induction, which we do by marking the lemma with the attribute `{:induction false}`. We can then write the proof as follows:

```
least lemma {:induction false} AscendingIsFinite(s: Stream<A>)
  requires (s == Nil || Acc(s.head)) && Ascending(s)
  ensures IsFinite(s)
{
  if s == Nil {
  } else {
    AscendingIsFinite(s.tail);
  }
}
```

This proof straightforwardly introduces two cases: either `s == Nil` or `s != Nil`. In the first case, `IsFinite(s)` follows directly. In the second case, we call the lemma recursively to obtain `IsFinite(s.tail)`, from which `IsFinite(s)` follows. Calling a lemma recursively is what in mathematics we usually refer to as "invoking the induction hypothesis".

We can add more details to this proof by considering whether or not `s.tail == Nil`. If it is, we can easily justify calling the induction hypothesis. If it is not, we unfold the definitions of `Ascending` and `Acc` before we call the induction hypothesis. We then have:

```
least lemma {:induction false} AscendingIsFinite(s: Stream<A>)
  requires (s == Nil || Acc(s.head)) && Ascending(s)
  ensures IsFinite(s)
{
  if s == Nil {
  } else if s.tail == Nil {
    AscendingIsFinite(s.tail);
  } else {
    assert lt(s.head, s.tail.head) && Ascending(s.tail); // by Ascending(s)
    assert forall y :: lt(s.head, y) ==> Acc(y); // by Acc(s.head)
    assert Acc(s.tail.head); // by the previous two lines
    AscendingIsFinite(s.tail);
  }
```

This manual proof is rather carefree. For example, what makes sure the recursive calls terminate? If we want to find the answer to that, we'll need to understand what makes a lemma a `least` lemma. Dafny turns such an extreme lemma into a *prefix lemma*, similar to how an extreme predicate is turned into a prefix predicate. The prefix lemma gives some scaffolding into which it places a rewritten version of the body we wrote for the extreme lemma. The following gives you an idea of what that looks like:

```
lemma {:induction false} AscendingIsFinite#[_k: ORDINAL](s: Stream<A>)
  requires (s == Nil || Acc#[_k](s.head)) && Ascending(s)
  ensures IsFinite(s)
{
  if _k.Offset != 0 {
    if s == Nil {
    } else if s.tail == Nil {
      AscendingIsFinite#[_k - 1](s.tail);
    } else {
      assert lt(s.head, s.tail.head) && Ascending(s.tail); // by Ascending(s)
      assert forall y :: lt(s.head, y) ==> Acc#[_k - 1](y); // by Acc(s.head)
      assert Acc#[_k - 1](s.tail.head); // by the previous two lines
      AscendingIsFinite#[_k - 1](s.tail);
    }
  } else {
    forall k' | k' < _k && (s == Nil || Acc#[k'](s.head)) && Ascending(s) {
      AscendingIsFinite#[k'](s);
    }
  }
}
```

Note that, like prefix predicates, the prefix lemma places the implicit `_k` parameter in square brackets. The "then" branch of the lemma's body applies when `_k` is a successor ordinal and the "else" branch applies when `_k` is a limit ordinal. The `forall` statement in the "else" branch has the effect of calling

`AscendingIsFinite#[k'](s)` on every `k'` that is both smaller than `_k` and satisfies the precondition (this is what Dafny's automatic induction does [3]). Finally, note that all occurrences of `Acc` and `AscendingIsFinite` in the body of the original `least lemma` have been automatically rewritten into `Acc#[_k - 1]` and `AscendingIsFinite#[_k - 1]`, respectively, and occurrences of `Acc` in the precondition have been rewritten into `Acc#[_k]`.

Dafny's syntax does not allow you to declare a lemma with square brackets like we showed up. But if we change the name of the lemma to, say, `AscendingIsFinite'`, rename `_k` to `k`, and declare `k` as an ordinary parameter, then we get a lemma that is syntactically valid:

```
lemma {:induction false} AscendingIsFinite'(k: ORDINAL, s: Stream<A>)
  requires (s == Nil || Acc#[k](s.head)) && Ascending(s)
  ensures IsFinite(s)
{
  if k.Offset != 0 {
    if s == Nil {
    } else if s.tail == Nil {
      AscendingIsFinite'(k - 1, s.tail);
    } else {
      assert lt(s.head, s.tail.head) && Ascending(s.tail); // by Ascending(s)
      assert forall y :: lt(s.head, y) ==> Acc#[k - 1](y); // by Acc(s.head)
      assert Acc#[k - 1](s.tail.head); // by the previous two lines
      AscendingIsFinite'(k - 1, s.tail);
    }
  } else {
    forall k' | k' < k && (s == Nil || Acc#[k'](s.head)) && Ascending(s) {
      AscendingIsFinite'(k', s);
    }
  }
}
```

This lemma verifies as well.

3.3. Iterates of `F`

The *iterates* of a function from an initial element `x` is a stream

```
x, F(x), F(F(x)), F^3(x), F^4(x), ...
```

The stream ends if the next element would be a repeat of the previous. That is, the stream is finite if and only if the last element is a fixpoint of the function. Here is a function for generating iterates:

```
function Iterates(x: A): Stream<A> {
  if x == F(x) then
    Cons(x, Nil)
  else
    Cons(x, Iterates(F(x)))
}
```

Dafny accepts this function without any complaints about termination. Indeed, the function results in an infinite stream if the iterates never reach a fixpoint. The reason is that the self-call is placed immediately inside the constructor of a codatatype. In this case, the self-call can be referred to as a *corecursive call* and Dafny deems it to be *productive*. Productivity ensures that the function definition is mathematically consistent, so Dafny is satisfied with the function definition even if the self-call never terminates.

In Dafny, the definition of a function can include both recursive and corecursive calls. (For example, see the `Filter` function in [4].) So, technically, being “corecursive” is a property of a call, not of a function.

At run time, a corecursive call is evaluated lazily, like functions in Haskell.

Let's prove our claim that the last element of a finite stream of iterates is a fixpoint. Because the antecedent of this lemma is a least predicate, we write the proof using a `least lemma`:

```
least lemma LastIterateIsFixpoint(x: A)
  requires IsFinite(Iterates(x))
  ensures IsFixpoint(Last(Iterates(x)))
{
  if Iterates(x).tail != Nil {
    LastIterateIsFixpoint(F(x));
  }
}
```

If `Iterates(x)` is a singleton stream, then the postcondition follows directly. Otherwise, the least lemma is called again with `F(x)` as the initial element of the iterates.

An easy mistake is to conclude from the monotonicity of `F` that the iterates of `F` are ascending. This may not be true, but it is true if the first two elements of the iterates are ascending. Let's state and prove this property. Since the conclusion contains a greatest predicate (`Ascending`), we'll be helped by using a `greatest lemma`:

```
greatest lemma IteratesAreAscending(x: A)
  requires le(x, F(x))
  ensures Ascending(Iterates(x))
{
  if x != F(x) {
    FMonotonic(x, F(x));
    IteratesAreAscending(F(x));
  }
}
```

Note how the automatic adaption of this lemma into the prefix lemma gives us the illusion of writing a proof without concern about termination. Yet, it will be instructive to write out this particular proof in more detail, because it will show a subtlety of the encoding of greatest lemmas. Here is the same lemma, but with more detail:

```
greatest lemma IteratesAreAscending(x: A)
  requires le(x, F(x))
  ensures Ascending(Iterates(x))
{
  if x != F(x) {
    calc {
      le(x, F(x));
    ==> { FMonotonic(x, F(x)); }
      le(x, F(x)) && le(F(x), F(F(x)));
    ==> { IteratesAreAscending(F(x)); }
      le(x, F(x)) && Ascending(Iterates(F(x)));
    ==> // def. Ascending
      Ascending#[_k](Iterates(x));
    }
  }
}
```

In the last line of this proof, we had to explicitly give the index to `Ascending#`. The reason is that Dafny rewrites every occurrence of `Ascending` in the body of given `greatest lemma` into `Ascending#[_k - 1]`. But in order for the last line to follow from the prior line, the index needs to be `_k` (since the prior line is the last line unfolded once). Moreover, when the postcondition is adapted for the prefix lemma, it is rewritten into

```
  ensures Ascending#[_k](Iterates(x))
```

The good thing is that the body of an extreme lemma allows us to use `_k` directly, so we can write `Ascending#[_k]` when we need it.

Remark 3.
To summarize this subtle point, suppose `P(x)` is a least predicate. Then, note that the rewrite into the prefix lemma uses `_k` in the specification and `_k - 1` in the body:

```
least lemma L(x: X)
  requires P(x) // adapted for prefix lemma by automatic rewrite into P#[_k]
  // ...
{
  assert P(x); // adapted for prefix lemma by automatic rewrite into P#[_k - 1]
  // ...
}
```

Dually, if `Q(x)` is a greatest predicate, then we have:

```
greatest lemma G(x: X)
  // ...
  ensures Q(x) // adapted for prefix lemma by automatic rewrite into Q#[_k]
{
  // ...
  assert Q(x); // adapted for prefix lemma by automatic rewrite into Q#[_k - 1]
}
```

As a final lemma in this section, we combine the `IteratesAreAscending` and `AscendingIsFinite` to prove

```
lemma IteratesAreFinite(x: A)
  requires Acc(x) && le(x, F(x))
  ensures IsFinite(Iterates(x))
{
  IteratesAreAscending(x);
  AscendingIsFinite(Iterates(x));
}
```

Note that this is an ordinary lemma (not an extreme lemma), since our proof simply uses other lemmas without any need to introduce the `_k` index.

3.4. Computing the Least Fixpoint

We've finally arrived at where we can write a function that computes the least fixpoint of `F`. We'll do it in two different way, first via our `Iterates` function:

```
function LFP(): A {
  AboutIteratesOfBot();
  Last(Iterates(bot))
}
```

The compiled function body is just `Last(Iterates(bot))`. However, to use function `Last`, we must meet its precondition that the given stream is finite. We gather that information by calling a lemma, which we state and prove thus:

```
lemma AboutIteratesOfBot()
  ensures IsFinite(Iterates(bot))
{
  BotAcc();
  BotSmallest(F(bot));
  IteratesAreFinite(bot);
}
```

Dafny accepts the `LFP()` function, and running it will terminate and return, uh, the last element of the finite stream `Iterates(bot)`. But what says that element is the least fixpoint? Indeed, what says a least fixpoint exists, let along any fixpoint at all? Here is a lemma that addresses all of those points:

```
lemma LFPIsLeastFixpoint()
  ensures var lfp := LFP();
    IsFixpoint(lfp) &&
    forall y :: IsFixpoint(y) ==> le(lfp, y)
```

Let's build up the proof.

First, just like in the postcondition of the lemma, it will be convenient in the proof to have a name for the result of `LFP()`. So, let's assign it to a local variable:

```
{
  var lfp := LFP();
```

We'll need to know, just like in the body of `LFP()` that the iterates from `bot` form a finite stream, so let's call the lemma that gives us this information:

```
  AboutIteratesOfBot();
```

Next, let's prove `lfp` to be a fixpoint. We'll structure this part of the proof using an `assert by` statement, which first states what we intend to prove and then, in curly braces, gives the proof:

```
  assert IsFixpoint(lfp) by {
    LastIterateIsFixpoint(bot);
  }
```

This shows that a fixpoint of `F` exists, because `LFP()` returns one.

The other proof obligation is to show the universal quantification

```
forall y :: IsFixpoint(y) ==> le(lfp, y)
```

We'll use the “universal introduction” rule from logic, which says that proving the body of the quantifier for an arbitrary `y` is enough to establish it for all `y`. In Dafny, universal introduction is done using a `forall` statement:

```
  forall y | IsFixpoint(y)
    ensures le(lfp, y)
  {
    // proof of le(lfp, y) goes here...
  }
}
```

All we have left to do now is fill in the body of the `forall` statement. The basic idea of this proof is to follow each successive element `x` of `Iterates(bot)` and show, for each one, that `le(x, y)`. Let's write the proof in two different ways.

3.5. A Proof by Recursion

Our first proof uses an auxiliary lemma that we prove by induction. That is, the lemma calls itself recursively to form a proof. The core specification of the lemma is

```
lemma IteratesDon'tSkipAnyFixpoint(x: A, y: A)
  requires IsFixpoint(y)
  ensures le(Last(Iterates(x)), y)
```

which would let us call the lemma as `IteratesDon'tSkipAnyFixpoint(bot, y)` from inside the `forall` statement of `LFPIsLeastFixpoint()`. But there are three problems with this specification. You would discover these if you'd attempt the proof, but we'll just state them here:

The first problem is that calling `Last` requires its argument to be finite. The simplest solution to this problem is to add `IsFinite(Iterates(x))` as a lemma precondition.

The second problem is that the lemma doesn't hold for an arbitrary `x`. It can hold only if `x` is below `y` to begin with. To correct this, we add the precondition `le(x, y)`.

The third problem is that we'll need a termination metric for the lemma's recursive call. Since the basic idea of the proof is to follow the successive iterates, we'll use the length of the finite iterate stream as the decreasing measure.

Here's the result:

```
lemma IteratesDon'tSkipAnyFixpoint(x: A, y: A)
  requires IsFinite(Iterates(x))
  requires le(x, y) && IsFixpoint(y)
  ensures le(Last(Iterates(x)), y)
  decreases Length(Iterates(x))
{
  if Iterates(x) == Cons(x, Nil) {
    // we're done
  } else {
    AboutLength(Iterates(x));
    FMonotonic(x, y);
    IteratesDon'tSkipAnyFixpoint(F(x), y);
  }
}
```

The “else” branch of the proof, which invokes the induction hypothesis, also needs some information about `Length` as well as the monotonicity of `F`.

To call this lemma from the `forall` statement in `LFPIsLeastFixpoint()`, we also need the information `le(bot, y)`, so we write

```
    BotSmallest(y);
    IteratesDon'tSkipAnyFixpoint(bot, y);
```

That concludes the proof and the definition of our iterator.

4. Alternative Solutions

4.0. A Proof by Iteration

Every programmer knows that many problems can be solved either recursively or iteratively (that is, using a loop). But not every mathematician exercises this same choice when it comes to proofs. To show how it's done, let's write an iterative proof of `LFPIsLeastFixpoint()`'s `forall` statement here. This proof will go inside the curly braces of the `forall` statement.

We'll use a local variable `ii` to hold the iterates that we have yet to loop over. To say that `ii` is indeed a stream of iterates, we use the loop invariant

```
ii == Iterates(ii.head)
```

Talking about `ii.head` requires knowing `ii != Nil`, so we'll add `ii != Nil` to the loop invariant, too. Like in the precondition of the recursive lemma in the previous section, we'll use

```
IsFinite(ii) && le(ii.head, y)
```

as part of the loop invariant, and we'll use `Length(ii)` as the measure that proves termination. Since we're writing the loop inside the `forall` statement, we don't need to repeat the condition `IsFixpoint(y)`, like we had to do in the recursive proof. Instead, we need to keep track of that `Last(ii)` remains equal to `lfp`, which is not necessary in the recursive lemma's postcondition, since that proof does not involve any variable that changes values.

All in all, our iterative version of `LFPIsLeastFixpoint` looks like this:

```
lemma LFPIsLeastFixpoint()
  ensures var lfp := LFP();
    IsFixpoint(lfp) &&
    forall y :: IsFixpoint(y) ==> le(lfp, y)
{
  var lfp := LFP();
  AboutIteratesOfBot();

  assert IsFixpoint(lfp) by {
    LastIterateIsFixpoint(bot);
  }

  forall y | IsFixpoint(y)
    ensures le(lfp, y)
  {
    BotSmallest(y);
    var ii := Iterates(bot);
    while ii.tail != Nil
      invariant ii != Nil && ii == Iterates(ii.head)
      invariant IsFinite(ii)
      invariant le(ii.head, y)
      invariant lfp == Last(ii)
      decreases Length(ii)
    {
      AboutLength(ii);
      FMonotonic(ii.head, y);
      ii := ii.tail;
    }
  }
}
```

4.1. Omitting the Stream of Iterates

We used streams in development, because it was convenient to define `Iterates(x)` without concern about termination. Function `LFP()` above computes `Iterates(bot)` and then selects its final element. Space-wise, this would be okay at run time, because the elements of the stream are computed lazily. For this reason, there's never a time when the running program needs to hold on to the stream of all iterates. For the same reason, streams are unnecessary for the running program. Let's write a function that (like `FindFixpoint` back in Section 0) computes a fixpoint without using streams.

```
function FindFixpoint(x: A): A
  requires IsFinite(Iterates(x))
  decreases Length(Iterates(x))
{
  var y := F(x);
  if x == y then
    x
  else
    AboutLength(Iterates(x));
    FindFixpoint(y)
}
```

As you can see, this function uses `Iterates` in its proof of termination, but doesn't use `Iterates` for any non-ghost purpose. Moreover, the function is tail recursive, so Dafny will compile it efficiently into a loop.

Let's not forget to prove the connection between `FindFixpoint` and `LFP()`. We'll start with a lemma that shows that `FindFixpoint(x)` does indeed follow `Iterates(x)`:

```
lemma FindFixpointFollowsIterates(x: A)
  requires IsFinite(Iterates(x))
  ensures FindFixpoint(x) == Last(Iterates(x))
  decreases Length(Iterates(x))
{
  if Iterates(x).tail != Nil {
    AboutLength(Iterates(x));
    FindFixpointFollowsIterates(F(x));
  }
}
```

With that lemma in hand, here's our final function for computing the least fixpoint:

```
function FindLeastFixpoint(): (lfp: A)
  ensures IsFixpoint(lfp)
  ensures forall y :: IsFixpoint(y) ==> le(lfp, y)
{
  AboutIteratesOfBot();
  FindFixpointFollowsIterates(bot);
  LFPIsLeastFixpoint();
  FindFixpoint(bot)
}
```

5. Conclusion

If you've forgotten the motivation for our main theorem, review the first few paragraphs of Section 0. Some of the words in those paragraphs may make more sense now.

Our tutorial journey took us through the definition and use of `codatatypes`, `least predicates` and `greatest predicates` (together known as *extreme* predicates), and ordinals. We saw how such definitions are automatically adapted into corresponding definitions of prefix predicates. It's most convenient when a program can use the extreme predicates directly, but the prefix predicates are also available for when they are needed.

Establishing the truth of a least predicate and making use of a greatest predicate are the same as for any predicate. Making use of a least predicate and establishing the truth of a greatest predicate are done via prefix predicates, for which a `least lemma` and `greatest lemma`, respectively, provides the scaffolding. The proof of an extreme lemma looks like a proof where the author forgot to worry about termination. Kozen and Silva have argued for a similar, carefree way of writing coinductive proofs [2]. Dafny adapts extreme lemmas into prefix lemmas using some rewriting. Fragments of this rewriting can be seen as hover text in the Dafny IDE.

Our journey also made use of a `codatatype`, whose values can be infinite structures. We started off by writing some useful functions on and lemmas about possibly infinite streams.

We also saw a number of tricks and techniques. For examples, we declared a function `PickK` to make sure the definition and proofs about `Length` used the same textual occurrences of `:|` (since each occurrence gives rise to its own Hilbert's-ε operator). We used a function `ReduceK` to combine the successor-ordinal and limit-ordinal cases in a compiled function. And we used a loop to write a proof, as an alternative to using recursion.

As a final remark, if you have previous experience using other proof systems, you will notice a difference in what a least predicate is. Other systems tend to define a least predicate as an inductive data structure that "remembers" how the least-predicate proof term was constructed. Thus, a lemma whose antecedent is a least predicate is done by induction over that data structure. In Dafny, a least predicate has no associated data (unless the user defines a separate data structure for this, using a `datatype` for a least predicate or a `codatatype` for a greatest predicate). The only clue to how a least predicate was established is the index `_k` of the prefix predicate, which gives (an ordinal) bound on the size of that derivation. One consequence of this design is that extreme predicates can be used as any other ghost function in a program.

A. Answer to Exercises

Answer to Exercise 0

Here is the (dubious) definition of `IsFinite'` as a greatest predicate:

```
greatest predicate IsFinite'(s: Stream) {
  s == Nil || IsFinite'(s.tail)
}
```

To show that `IsFinite'(s)` holds for any stream `s`, we declare the following greatest lemma:

```
greatest lemma IsFinite'True(s: Stream)
  ensures IsFinite'(s)
{
}
```

The body is empty, because Dafny's automatic induction completes the proof.

Answer to Exercise 1

To define `FiniteAndAscending`, use a least predicate:

```
least predicate FiniteAndAscending(s: Stream<A>) {
  match s
  case Nil => true
  case Cons(x, Nil) => true
  case Cons(x, Cons(y, _)) => lt(x, y) && FiniteAndAscending(s.tail)
}
```

References

[0]P. Cousot and R. Cousot. Constructive versions of Tarski's fixed point theorems. *Pacific Journal of Mathematics*, 81(1):43–57, 1979.

[1]Patrick Cousot and Rhadia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conference Record of the Fourth Annual ACM Symposium on Principles of Programming Languages*, pages 238–252. ACM, January 1977.

[2]Dexter Kozen and Alexandra Silva. Practical coinduction. *Mathematical Structures in Computer Science*, 27(7):1132–1152, 2017.

[3]K. Rustan M. Leino. Automating induction with an SMT solver. In Viktor Kuncak and Andrey Rybalchenko, editors, *Verification, Model Checking, and Abstract Interpretation — 13th International Conference, VMCAI 2012*, volume 7148 of *Lecture Notes in Computer Science*, pages 315–331. Springer, January 2012.

[4]K. Rustan M. Leino. Automating theorem proving with SMT. In Sandrine Blazy, Christine Paulin-Mohring, and David Pichardie, editors, *Interactive Theorem Proving — 4th International Conference, ITP 2013*, volume 7998 of *Lecture Notes in Computer Science*, pages 2–16. Springer, July 2013.

[5]K. Rustan M. Leino. Compiling Hilbert's epsilon operator. In Ansgar Fehnker, Annabelle McIver, Geoff Sutcliffe, and Andrei Voronkov, editors, *LPAR-20. 20th International Conferences on Logic for Programming, Artificial Intelligence and Reasoning — Short Presentations*, volume 35 of *EPiC Series in Computing*, pages 106–118. EasyChair, 2015.

[6]K. Rustan M. Leino. Dafny power user: Calling lemmas automatically. Manuscript KRML 265, June 2019. http:// leino. science/ papers/ krml265. html.

[7]K. Rustan M. Leino and Nadia Polikarpova. Verified calculations. In Ernie Cohen and Andrey Rybalchenko, editors, *Verified Software: Theories, Tools, Experiments — 5th International Conference, VSTTE 2013, Revised Selected Papers*, volume 8164 of *Lecture Notes in Computer Science*, pages 170–190. Springer, 2014.

[8]Per Martin-Löf. *Notes on Constructive Mathematics*. Stockholm, Almqvist & Wiksell, 1970.

[9]A. Tarski. A lattice theoretical fixed point theorem and its applications. *Pacific Journal of Mathematics*, 5:285–309, 1955.